

Task : Bash Scripting

Name: Anjelina Behera

Course : CyberSecurity

Date: 23-07-26

1. Objective -

The objective of this task was to write a Bash script inside `read_secret.sh` that reads the contents of `secret.txt`, stores that content in a variable, and prints the value of that variable to the terminal which reveals the contents of `secret.txt`.

2. Prerequisites – Why Use a Variable Instead of Just cat?

This assignment specifically requires storing the file's content in a variable before printing it, rather than simply running `cat secret.txt` directly. This is **command substitution** - a core Bash scripting concept where the output of a command is captured and assigned to a variable using the `$(...)` syntax.

3. Initial Steps

Before writing the script, I explored the working directory to understand the file structure and permissions I was working with :

```
student@hacking:~$ ls
```

```
loot  practice  read_secret.sh  run_secret
secret.txt
```

Running `ls -la` on the home directory showed the following permissions:

```
-rwxrwxrwx 1 root  student  102  read_secret.sh
-rwsr-xr-x 1 root   root    16008 run_secret
-rw-r----- 1 root   perm      26  secret.txt
```

Interpretation of these permissions:

- `secret.txt` — owned by root, group perm. Permissions are `rw-r-----`, meaning only the owner (root) can read/write it, and only members of the perm group have read access. As the student user, I belong to neither, so direct access is denied.

- `read_secret.sh` — owned by root but it is readable, writable and executable (`rw-rw-rw-`), meaning any user, including student, can edit its contents.
- `run_secret` — owned by root:root with permissions `rwsr-xr-x`. The `s` in place of the owner's execute bit indicates the **SUID (Set User ID)** bit is set. This means that whenever this binary is executed, it runs with the privileges of its owner (root) regardless of which user actually runs it.

4. Step-by-Step Process

Step 1: Attempting Direct Access

As expected from the permission analysis above, direct access as the student user was denied, I also attempted to work around this by changing the file's permissions directly:

```
student@hacking:~$ cat secret.txt
cat: secret.txt: Permission denied
student@hacking:~$ chmod +r secret.txt
chmod: changing permissions of 'secret.txt':
Operation not permitted
```

Step 2: Editing `read_secret.sh`

Since `read_secret.sh` was writable, I opened it in a text editor and wrote the following script :

```
student@hacking:~$ nano read_secret.sh
#!/bin/bash
content=$(cat secret.txt)
echo "$content"
```

Interpretation :

- `#!/bin/bash` - the shebang line, which tells the system to interpret this script using the Bash shell.
- `content=$(cat secret.txt)` - this runs `cat secret.txt` and captures its output, storing it inside the variable `content` using command substitution (`$ ( ... )`).
- `echo "$content"` prints the value stored in the variable to the terminal.

I saved the file and exited the editor (`ctrl+X` , `Y` , `Enter` in nano).

Step 3: Running the Script Directly :

Running `read_secret.sh` directly as the `student` user failed to read `secret.txt`, because a script executes with the privileges of the user who runs it — not the user who owns the script file. Since `secret.txt` was unreadable by `student` regardless of who wrote the script, the `cat` command inside the script still hit a `Permission denied` error.

That is where `run_secret` came in.

Step 4: Executing via `run_secret`

```
student@hacking:~$ ./run_secret
```

```
flag{you_wrote_an_script}
```

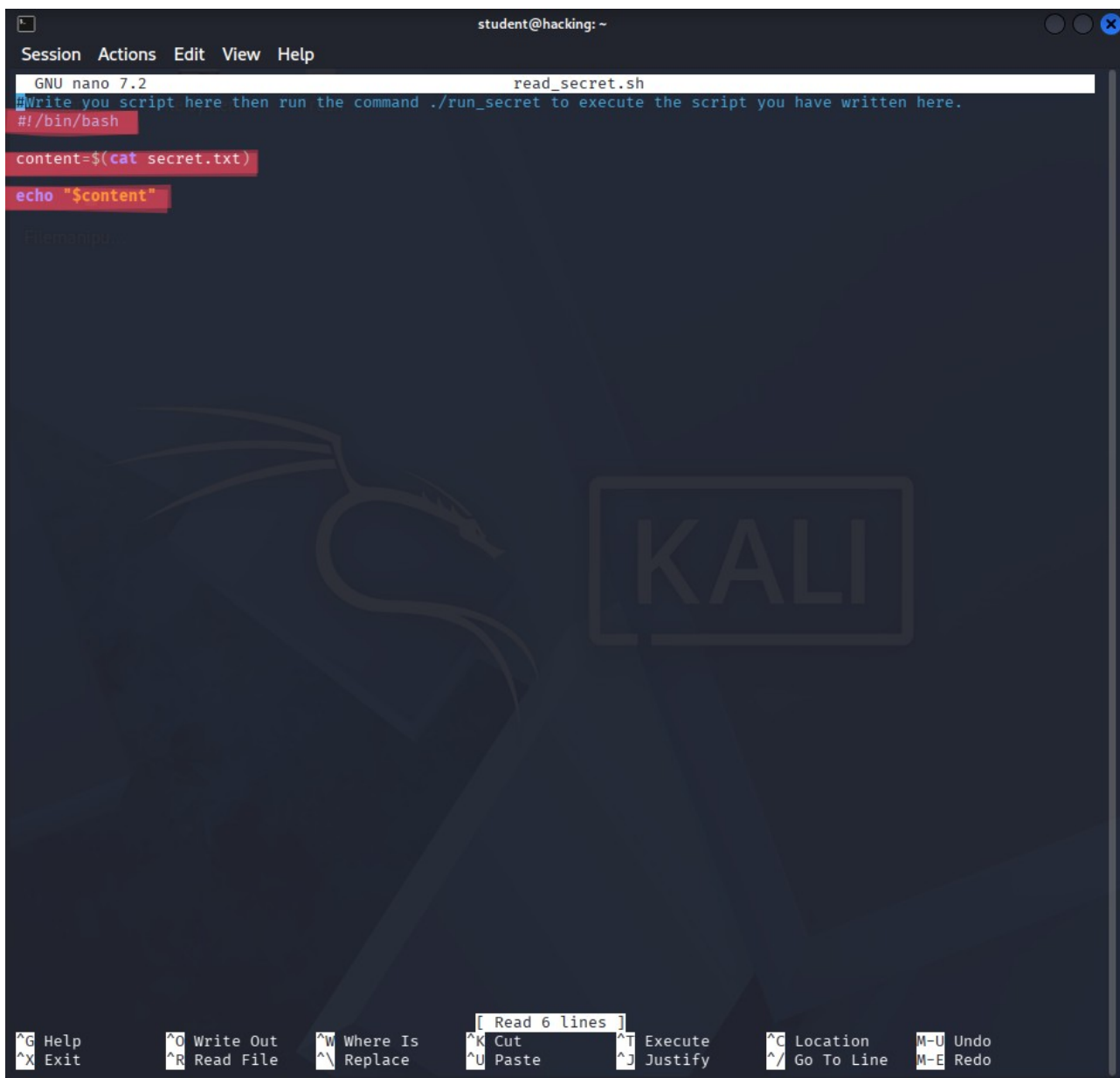
Because `run_secret` has the **SUID bit** set and is owned by `root`, executing it runs its underlying logic with `root` privileges — regardless of the fact that `student` invoked it. This allowed the process to successfully read `secret.txt` and output its contents.

Contents of `secret.txt`:

```
flag{you_wrote_an_script}
```

Step 5: Screenshot Evidence

```
student@hacking:~$ cat secret.txt
cat: secret.txt: Permission denied
student@hacking:~$ ls -la
total 64
drwxr-x— 6 student perm    4096 Apr 13 11:10 .
drwxr-xr-x 5 root    root    4096 Feb 26 13:57 ..
-rw— 1 student student    16 Jul 21 11:39 .bash_history
-rw-r--r-- 1 student student    220 Mar 31 2024 .bash_logout
-rw-r--r-- 1 student student   3771 Mar 31 2024 .bashrc
drwx— 2 student student    4096 Feb 20 17:19 .cache
drwxrwxr-x 3 student student    4096 Jan 28 06:04 .local
drwxrwxr-x 2 student student    4096 Jan 28 06:05 loot
drwxrwxr-x 2 student student    4096 Jan 28 06:05 practice
-rw-r--r-- 1 student student    807 Mar 31 2024 .profile
-rwxrwxrwx 1 root    student    102 Apr 13 09:53 read_secret.sh
-rwsr-xr-x 1 root    root      16008 Apr 13 09:39 run_secret
-rw-r— 1 root    perm      26 Feb 19 16:47 secret.txt
student@hacking:~$ chmod +r secret.txt
chmod: changing permissions of 'secret.txt': Operation not permitted
student@hacking:~$ nano read_secret.sh
student@hacking:~$ ./run_secret
flag{you_wrote_an_script}
student@hacking:~$
```



```
student@hacking: ~
Session Actions Edit View Help
GNU nano 7.2 read_secret.sh
#Write you script here then run the command ./run_secret to execute the script you have written here.
#!/bin/bash
content=$(cat secret.txt)
echo "$content"

[ Read 6 lines ]
^G Help      ^O Write Out  ^W Where Is   ^K Cut        ^T Execute    ^C Location   M-U Undo
^X Exit      ^R Read File  ^_ Replace    ^U Paste      ^J Justify    ^_ Go To Line  M-E Redo
```

The screenshot displays a Kali Linux terminal window on the left and a task interface on the right. The terminal shows system information, update status, and file permissions. The task interface shows a series of questions and answers related to the terminal output.

Terminal Output:

```
student@hacking:~$ cat /etc/passwd
root:x:0:0:root:/root:/bin/bash
daemon:x:1:1:daemon:/usr/sbin:/usr/sbin/nologin
bin:x:2:2:bin:/bin:/usr/sbin/nologin
systemd:x:3:3:systemd:/var/lib/containers:/usr/sbin/nologin
ubuntu:x:1000:1000:ubuntu:/home/ubuntu:/usr/bin/zsh
student:x:1001:1001:student:/home/student:/usr/bin/zsh

student@hacking:~$ ls -la /home/student/.cache
total 64
drwxr-xr-x 5 student perm 4096 Jul 23 08:16 .
drwxr-xr-x 5 root root 4096 Feb 26 13:57 ..
-rw-r--r-- 1 student student 284 Jul 23 08:16 .bash_history
-rw-r--r-- 1 student student 228 Mar 31 2024 .bash_logout
-rw-r--r-- 1 student student 3771 Mar 31 2024 .bashrc
drwxr-xr-x 2 student student 4096 Feb 20 17:19 .cache
drwxr-xr-x 3 student student 4096 Jan 28 06:04 .local
drwxr-xr-x 2 student student 4096 Jan 28 06:05 .loot
drwxr-xr-x 2 student student 4096 Jan 28 06:05 .practice
-rw-r--r-- 1 student student 887 Mar 31 2024 .profile
-rwxrwxr-x 1 root student 158 Jul 23 08:13 read_secret.sh
-rwsr-xr-x 1 root root 16088 Apr 13 09:39 run_secret
-rw-r--r-- 1 root perm 26 Feb 19 16:47 secret.txt

student@hacking:~$ nano read_secret.sh
student@hacking:~$ ./run_secret
flag{you_wrote_an_script}
```

Task Interface:

readme.txt ✓ Correct Answer

What are the contents of the file which was in the "loot" folder?

flag{you_know_cd_is_cat_now} ✓ Correct Answer

What are the contents of the hidden file in the "loot" folder?

flag{you_found_the_hidden_file} ✓ Correct Answer

Understand how permissions work in linux and and read the content of /home/student/practice/restricted_file.txt

flag{you_understood_permissions} ✓ Correct Answer

Use any Text Editor and edit the content of the read_secret.sh. Inside that file, write a script to read the contents of secret.txt. What is inside secret.txt?

flag{you_wrote_an_script} ✓ Correct Answer

Task List:

- Task 3: Stages of Hacking
- Task 4: Information Gathering Active and Passive
- Task 5: Web App Hacking
- Task 6: VAPT Scan
- Task 7: Investigate an Incident
- Task 8: Layer 2 Traffic Analysis

The SUID bit and its risks :

The SUID (Set User ID) permission bit allows a program to run with the privileges of its *owner*, rather than the user executing it. This is normally used for legitimate purposes. However, if a SUID-root binary calls out to a script or file that is **writable by a non-root user** as `read_secret.sh` was in this task, it creates a privilege escalation vulnerability. An attacker can modify the writable script, and when the SUID binary executes it, the malicious or modified code runs with root privileges.

Command substitution as a scripting fundamental:

This task reinforced the practical use of `$ ( . . . )` for capturing command output into a variable, a pattern used throughout real-world Bash scripting for logging, validation, and conditional logic, rather than just displaying output once.

5. Conclusion

This task required writing a Bash script to read the contents of a permission-restricted file and print it via a variable. While the script itself was straightforward, the underlying challenge illustrated an important real-world security concept: how a SUID-root binary combined with a world-writable script can be leveraged to bypass file permission restrictions — in this case, revealing the flag `flag{you_wrote_an_script}`. This reinforced both basic Bash

scripting syntax (command substitution, variables, echo) and a foundational Linux privilege escalation concept.